



Docker & Containerization

Docker Basics • Dockerfile • Volume • Network • Multi-Stage • Compose • CI/CD

By: Pranpaveen Laykaviriyakul

ทำไมต้องใช้ Docker?

✗ ปัญหา (ก่อนมี Docker)

"It works on my machine!"

- ติดตั้ง dependencies แต่ละเครื่องต่างกัน
- รันได้บน dev แต่ crash บน server
- ต้องตั้งค่า environment ใหม่ทุกครั้ง
- Version conflict ระหว่าง projects

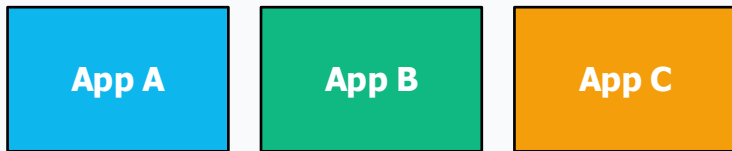
✓ Docker แก้ปัญหา

"Works everywhere!"

- Package app + dependencies ใน Container
- รันเหมือนกันทุก environment
- Deploy ง่าย ไม่ต้องตั้งค่าซ้ำ
- Isolate แต่ละ project ได้

Container คืออะไร?

Container = กล่องที่บรรจุ Application พร้อม dependencies ทั้งหมด
ทำงานแบบ isolated และรันได้บนทุก environment ที่มี Docker



เหมือนตู้คอนเทนเนอร์ขนส่งสินค้า
→ ขนได้ทุกที่ ไม่สนว่าเรือ รถ หรือเครื่องบิน

 มี code, runtime, libraries ครบ

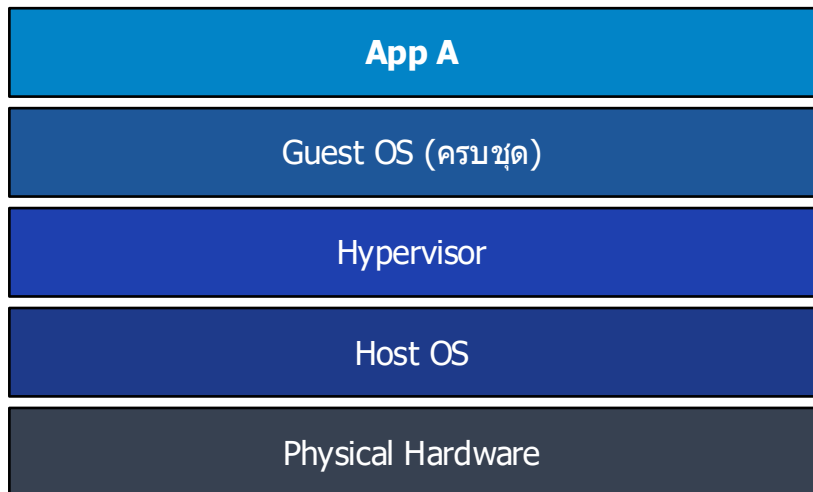
 เบากว่า VM (shares OS kernel)

 Isolated จาก container อื่น

 Reproducible ทุก environment

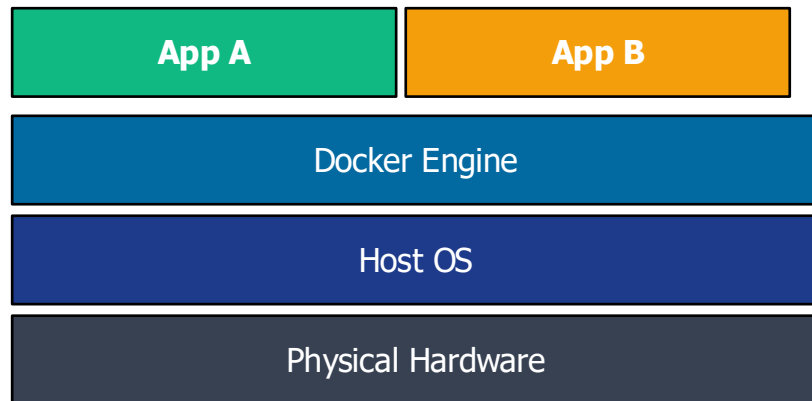
Container vs Virtual Machine

Virtual Machine (VM)



 **หนัก | ช้า | RAM มาก | Boot นาน**

Docker Container



 **เบา | เร็ว | ประหยัด | Start ทันที**

ติดตั้ง Docker

Docker Desktop

<https://www.docker.com/products/docker-desktop/>



Windows

Windows 10/11
(WSL2 required)



macOS

Intel &
Apple Silicon



Linux

Ubuntu, Debian
Fedora, etc.

ตรวจสอบการติดตั้ง:

```
docker --version  
docker run hello-world
```

Docker Basic Commands

Command	Description
<code>docker pull <image></code>	ดึง image จาก Docker Hub
<code>docker run <image></code>	รัน container จาก image
<code>docker run -d -p 8080:80 <image></code>	รันแบบ background + map port (host:container)
<code>docker ps</code>	แสดง container ที่รันอยู่
<code>docker ps -a</code>	แสดงทุก container รวม stopped
<code>docker stop <id></code>	หยุด container
<code>docker rm <id></code>	ลบ container
<code>docker images</code>	แสดง image ทั้งหมด
<code>docker rmi <image></code>	ลบ image
<code>docker logs <id></code>	ดู logs ของ container
<code>docker exec -it <id> bash</code>	เข้าไปใน container แบบ interactive

Docker Image vs Container



Docker Image

เหมือน **"Blueprint"** หรือ **"แม่พิมพ์"**

- Read-only template
- สร้างจาก Dockerfile
- เก็บใน Docker Hub / Registry
- สร้าง Container ได้หลายตัว



Docker Container

เหมือน **"บ้าน"** ที่สร้างจากแม่พิมพ์

- Instance ที่รันได้จาก Image
- มี writable layer เป็นของตัวเอง
- Start / Stop / Delete ได้
- แต่ละตัว isolate จากกัน



Image = สูตรพิซซ่า / Container = พืชชำที่อบแล้ว

SECTION

02

Dockerfile

เขียน Blueprint สำหรับสร้าง Docker Image ของคุณเอง

Dockerfile คืออะไร?

Dockerfile = ไฟล์ Text ธรรมดาที่บอก Docker ที่จะขั้นตอนว่าต้องทำอะไรเพื่อสร้าง Image เหมือน "สคริปต์ทำอาหาร" ที่บอกวิธีประกอบ app ของเรา



จุดสำคัญที่ต้องรู้:

- ชื่อไฟล์ต้องเป็น Dockerfile (ไม่มี extension)
- แต่ละบรรทัดคือ Instruction (FROM, RUN, COPY, ...)
- Docker build ที่ละ layer แล้ว cache ไว้ → build ครั้งต่อไปเร็วขึ้น
- ยิงรวม RUN หลายคำสั่งได้ → image ยิ่งเล็ก

Dockerfile Instructions

FROM	Base image ที่ใช้เป็นจุดเริ่มต้น	<code>FROM node:18-alpine</code>
WORKDIR	กำหนด working directory ใน container	<code>WORKDIR /app</code>
COPY	Copy ไฟล์จาก host เข้า container	<code>COPY . .</code>
RUN	รัน shell command ตอน build image	<code>RUN npm install</code>
ENV	กำหนด environment variable	<code>ENV NODE_ENV=production</code>
EXPOSE	ประกาศ port ที่ container จะใช้งาน	<code>EXPOSE 3000</code>
CMD	คำสั่งที่รันเมื่อ container เริ่มทำงาน	<code>CMD ["node", "app.js"]</code>

ตัวอย่าง Dockerfile (Node.js App)

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "app.js"]
```

← เลือก image ที่เล็กที่สุด (alpine)

← กำหนด working directory

← Copy package.json ก่อน
→ ให้ npm layer ถูก cache

← Copy source code ทีหลัง
(เพื่อประหยัด rebuild time)

← ประกาศ port ที่ app ใช้

← คำสั่งเริ่ม app



Build: `docker build -t myapp .` | Run: `docker run -p 3000:3000 myapp`

Build & Run Docker Image

1. Build Image

```
docker build -t myapp:1.0 .  
# -t = ชื่อ (tag) ของ image | . = ใช้ Dockerfile ในโฟลเดอร์ปัจจุบัน
```

2. Run Container

```
docker run -d -p 3000:3000 --name myapp myapp:1.0  
# -d = detach (background) | -p host:container | --name = ตั้งชื่อ container
```

3. Manage Container

```
docker ps          # ดู containers ที่รันอยู่  
docker stop myapp  # หยุด | docker rm myapp # ลบ
```

.dockerignore — ลด Image Size

.dockerignore บอก Docker ว่าไฟล์ไหนที่ไม่ต้อง COPY เข้า image ช่วยลด build context size, เร่ง build และป้องกัน secret หลุด

ผลที่ได้:

Build เร็วขึ้น

Docker ส่งเฉพาะไฟล์ที่จำเป็น
เป็น build context ไปยัง daemon

Image เล็กลง

ไม่มี node_modules หรือ
ไฟล์ไม่จำเป็นอยู่ใน image

ปกป้องภัยขึ้น

.env / secret files
ไม่ถูกฝังเข้าไปใน image

```
# ตัวอย่าง .dockerignore

# Dependencies (ติดตั้งใหม่ใน image อยู่แล้ว)
node_modules/

# Git history
.git/
.gitignore

# Logs และ temp files
logs/
*.log
tmp/

# Environment & Secrets ← สำคัญมาก!
.env
.env.*

# Test files
**/*.test.js
**/*.spec.js
coverage/

# Docs
README.md
docs/
```

ENV & ARG — Environment Variables ใน Docker

Docker มีตัวแปร 2 ประเภทที่ใช้ต่างกัน:

ENV — Runtime Variable

```
ENV NODE_ENV=production
ENV PORT=3000

# override ตอน run:
docker run -e PORT=8080 myapp
```

- คงอยู่ตอน container รัน
- สืบทอดไปยัง child process
- override ได้ด้วย -e flag
- ⚠ อย่าใส่ secret ใน ENV Dockerfile!

ARG — Build-time Variable

```
ARG NODE_VERSION=20
FROM node:${NODE_VERSION}

# สั่งค่าตอน build:
docker build --build-arg
  NODE_VERSION=18 .
```

- มีผลแค่ตอน docker build
- ไม่มีใน container เมื่อรัน
- เหมาะสำหรับ version/config
- ✅ ปลอดภัยกว่า ENV สำหรับ secret ที่ใช้แค่ตอน build

SECTION

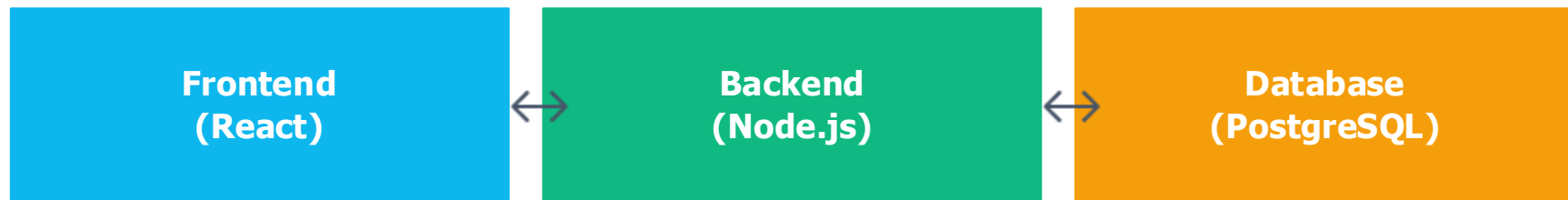
03

Docker Compose

จัดการ Multi-Container Application ด้วยไฟล์เดียว

Docker Compose คืออะไร?

Docker Compose = เครื่องมือสำหรับ define และ run multi-container apps ด้วยไฟล์ YAML ไฟล์เดียว (docker-compose.yml)



`docker-compose.yml` ← จัดการทุก service ในไฟล์เดียว

- ✓ สั่ง docker compose up เดียว ขึ้นทุก service พร้อมกัน
- ✓ Config network และ volume ได้ง่ายในที่เดียว
- ✓ เหมาะมากสำหรับ local development environment

ตัวอย่าง docker-compose.yml

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=db
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

← กำหนด services ทั้งหมด

← Build จาก Dockerfile

← Map port host:container

← รอ db พร้อมก่อนเริ่ม app

← ใช้ official postgres image

← Environment variables

← Persistent volume สำหรับข้อมูล

Docker Compose Commands

Command	Description
<code>docker compose up</code>	สร้างและ เริ่ม containers ทั้งหมด
<code>docker compose up -d</code>	เริ่มแบบ background (detach)
<code>docker compose down</code>	หยุดและลบ containers, networks
<code>docker compose down -v</code>	หยุด + ลบ volumes ด้วย
<code>docker compose ps</code>	แสดงสถานะของทุก service
<code>docker compose logs</code>	ดู logs ของทุก service
<code>docker compose logs -f app</code>	ดู live logs ของ service 'app'
<code>docker compose build</code>	Build images ใหม่ทั้งหมด
<code>docker compose exec app bash</code>	เข้าไปใน service container
<code>docker compose restart</code>	Restart ทุก service

Docker Volumes — ข้อมูล Persistent

Container เป็น ephemeral — ข้อมูลหายทันทีเมื่อลบ container | Volume คือกลไกเก็บข้อมูลให้คงอยู่ข้ามการสร้าง/ลบ container

Named Volume

`volumes: pgdata:`

Docker จัดการ path ให้อัตโนมัติ
เหมาะสำหรับ database



แนะนำ

Bind Mount

`-v $(pwd):/app`

Map folder host → container
เหมาะสำหรับ dev hot-reload



ขึ้นกับ path บน host

tmpfs Mount

`--tmpfs /tmp`

เก็บใน RAM ชั่วคราว
เร็วมาก แต่หายเมื่อ stop
เหมาะสำหรับ temp/sensitive data

```
docker volume create mydata
```

```
# สร้าง named volume
```

```
docker volume ls
```

```
# แสดงทุก volume
```

```
docker run -v mydata:/app/data myapp
```

```
# ใช้ volume กับ container
```

```
docker volume rm mydata
```

```
# ลบ volume
```

Docker Volumes — ตัวอย่างจริง

Named Volume — ข้อมูล persist แม้ลบ container

```
# 1. รัน postgres พร้อม named volume
docker run -d --name db \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15-alpine

# 2. ลบ container — volume ยังอยู่!
docker stop db && docker rm db
docker volume ls    # ← pgdata ยังแสดงอยู่

# 3. สร้างใหม่ — ข้อมูลเดิมครบ ✅
docker run -d --name db \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15-alpine
```

Bind Mount — Hot Reload สำหรับ Dev

```
# รัน container พร้อม bind mount
docker run -d -p 8000:8000 \
  -v $(pwd)/src:/app \
  fastapi-app

# แก้ไข src/main.py ✎
# FastAPI --reload detect การเปลี่ยนแปลง
# → ไม่ต้อง rebuild หรือ restart!

# ทดสอบผลลัพธ์
curl http://localhost:8000
```

container
รันอยู่



docker rm
(ลบ container)



pgdata
volume ยังอยู่



docker run
(container ใหม่)



ข้อมูลเดิม
ครบ ✅

Docker Network — การสื่อสารระหว่าง Container

Docker สร้าง virtual network ให้ container สื่อสารกัน
ใน docker-compose service ต่างๆคุยกันด้วยชื่อ service เป็น hostname อัตโนมัติ

bridge (default)

Default network
containers คุยกันผ่าน IP
ไม่มี DNS by service name

user-defined (bridge)

✅ แนะนำใน compose
DNS ด้วยชื่อ service อัตโนมัติ
Isolation ที่ดีกว่า

host (shared)

Share network stack
กับ host machine
ไม่มี network isolation

none (isolated)

ปิด network ทั้งหมด
Isolate สมบูรณ์แบบ
ใช้ใน security-sensitive tasks

```
docker network ls           # แสดงทุก network
docker network create mynet  # สร้าง custom network
docker run --network mynet myapp  # เชื่อม container กับ network
# ใน docker-compose: services คุยกันด้วย service name อัตโนมัติ ✅
```

Docker Network — ตัวอย่างจริง

docker-compose.yml — Network อัตโนมัติ

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - redis
  redis:
    image: redis:7-alpine

# Docker สร้าง network อัตโนมัติ
# ชื่อ: <folder>_default
# services คุยกันด้วย service name
```

main.py — เชื่อม Redis ด้วย service name

```
import redis

# ✅ ใน Docker — ใช้ service name
r = redis.Redis(
    host="redis", # ← service name
    port=6379
)

# ❌ localhost ไม่ work ใน container
# r = redis.Redis(host="localhost")

@app.get("/count")
async def count():
    n = r.incr("visits")
    return {"visits": n}
```

fastapi-app :8000



my_project_default (Docker Network DNS)



redis :6379

Multi-Stage Build — Dockerfile 2 ขั้นตอน

แยก build stage (image ใหญ่) ออกจาก run stage (image เล็ก)

→ production image ไม่มี compiler / build tools — เล็กกว่า 20x และปลอดภัยกว่า

Stage 1 — Build (golang:1.22-alpine)

```
FROM golang:1.22-alpine AS builder
```

```
WORKDIR /app
```

```
COPY go.mod go.sum ./  
RUN go mod download
```

```
COPY . .  
RUN CGO_ENABLED=0 GOOS=linux \  
    go build -o server .
```



Stage 2 — Run (alpine:3.19)

```
FROM alpine:3.19
```

```
RUN apk --no-cache add \  
    ca-certificates
```

```
WORKDIR /root/
```

```
COPY --from=builder /app/server .
```

```
EXPOSE 8080
```

```
CMD ["/server"]
```

Image Size:

golang:1.22-alpine ~330 MB

alpine:3.19 ~15 MB  เล็กกว่า 22x!

SECTION

04

Docker + CI/CD

Automate Build, Test & Deploy ด้วย GitHub Actions

CI/CD Pipeline with Docker



ทำไม Docker ถึงสำคัญใน CI/CD:

-  Consistent Env — build บน CI runner เหมือน local ทุกครั้ง ไม่มี "works on my machine"
-  Artifact พร้อม Deploy — push image แล้ว pull ไป run บน server ได้ทันที
-  Fast Rollback — ถ้า deploy มีปัญหา แค่ run image version ก่อนหน้าได้เลย

GitHub Actions + Docker

```
name: Build and Push

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USER }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Build & Push
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: myuser/myapp:latest
```

ขั้นตอน:

① Trigger เมื่อ push ไป main

② Checkout code จาก repo

③ Login ด้วย Secret ที่
ตั้งค่าใน GitHub Settings

④ Build Dockerfile และ
Push ไป Docker Hub

💡 เพิ่ม tag ด้วย commit:
tags: myapp:\${ github.sha }

Workshop & Q&A

-  Build Docker image จาก app ที่ตัวเองมี
-  เขียน docker-compose.yml สำหรับ app + database
-  Setup GitHub Actions ให้ push image ไป Docker Hub

 docs.docker.com | hub.docker.com